

- Description

This module manages a smart air purifier system, ensuring that:

1. The PM2.5 concentration is continuously monitored to evaluate indoor air quality.
2. The fan speed is automatically adjusted based on the PM2.5 level—if the concentration is high, the fan speed increases to enhance air cleaning efficiency; if it is low, the fan speed decreases to save energy.
3. The filter usage hours are tracked, and if they exceed a predefined threshold, a filter replacement alert is issued.

This system helps maintain optimal indoor air quality, optimize energy consumption, and ensure timely filter replacement for sustained performance.

- Specification

module SmartAirPurifierControl

var pm25, minFanSpeed, maxFanSpeed, currentFanSpeed: real

var filterUsageHours, filterReplacementThreshold: real

process updateAirQuality(newPM25: real) returns (currentFanSpeed: real)

functional scenario form:

pm25 >= 75 && currentFanSpeed = maxFanSpeed, Fan Speed set to maximum.

||

pm25 <= 35 && currentFanSpeed = minFanSpeed, Fan Speed set to minimum.

||

35 < pm25 and pm25 < 75 && currentFanSpeed = minFanSpeed + (maxFanSpeed - minFanSpeed) * ((pm25 - 35) / (75 - 35)), Fan Speed calculated by linear interpolation.

end process

process updateFilterUsage(additionalHours: real) returns (filterUsageHours: real)

functional scenario form:

(filterUsageHours < filterReplacementThreshold) && output "Filter usage updated: " + filterUsageHours + " hours." Filter usage updated.

OR

(filterUsageHours ≥ filterReplacementThreshold) && output "Filter replacement needed. Total usage: " + filterUsageHours + " hours.", Filter replacement alert triggered.

end process

end module

- Java Code

```
package code;
public class c74_SmartAirPurifier {
    private double pm25;
```

```

    private double fanSpeed;
    private double minFanSpeed;
    private double maxFanSpeed;
    private double filterUsageHours;
    private double filterReplacementThreshold;
    public c74_SmartAirPurifier(double initialPM25, double minFanSpeed,
double maxFanSpeed,
                                double initialFilterUsage, double filterRep
lacementThreshold) {
        this.pm25 = initialPM25;
        this.minFanSpeed = minFanSpeed;
        this.maxFanSpeed = maxFanSpeed;
        this.filterUsageHours = initialFilterUsage;
        this.filterReplacementThreshold = filterReplacementThreshold;
    }
    public void updateAirQuality(double newPM25) {
        pm25 = newPM25;
        adjustFanSpeed();
        System.out.printf("Updated PM2.5: %.2f µg/m³, Fan Speed: %.2f%%\n", pm25, fanSpeed);
        if (filterUsageHours >= filterReplacementThreshold) {
            System.out.println("ALERT: Filter replacement needed.");
        }
    }
    private void adjustFanSpeed() {
        if (pm25 > 75) {
            fanSpeed = maxFanSpeed;
        } else if (pm25 < 35) {
            fanSpeed = minFanSpeed;
        } else {
            fanSpeed = minFanSpeed + (maxFanSpeed - minFanSpeed) * ((pm
25 - 35) / (75 - 35));
        }
    }
    public void updateFilterUsage(double additionalHours) {
        if (additionalHours < 0) {
            System.out.println("Invalid usage time. Must be positive.");
;
            return;
        }
        filterUsageHours += additionalHours;
        System.out.printf("Updated Filter Usage: %.2f hours\n", filterU
sageHours);
        if (filterUsageHours >= filterReplacementThreshold) {

```

```

        System.out.println("ALERT: Filter replacement needed.");
    }
}

public double getFanSpeed() {
    return fanSpeed;
}
public double getFilterUsageHours() {
    return filterUsageHours;
}
public static void main(String[] args) {
    c74_SmartAirPurifier purifier = new c74_SmartAirPurifier(50.0, 20.
0, 100.0, 100.0, 500.0);

    // testcase-VT:
    //     purifier.updateAirQuality(80.0);
    //     purifier.updateAirQuality(30.0);
    //     purifier.updateAirQuality(55.0);
    //     purifier.updateFilterUsage(200.0);
    //     purifier.updateFilterUsage(250.0);
    //     purifier.updateFilterUsage(-2.0);
    //     purifier.updateFilterUsage(550.0);
    //     purifier.updateAirQuality(55.0);

    // testcase-FT:
    //     purifier.updateAirQuality(1.6425793966244292E308);
    //     purifier.updateAirQuality(-1.7976931348623157E308);
    //     purifier.updateAirQuality(1.94906280227999996E289);
    //     purifier.updateAirQuality(-1.5861998248785138E308);
    //     purifier.updateAirQuality(0.0);
    //     purifier.updateAirQuality(3.511119404027329E306);
    //     purifier.updateAirQuality(1.3465074068968717E308);
    //     purifier.updateAirQuality(9.063142030601998E290);

    // testcase-Z3:
    //     purifier.updateAirQuality(80.0);
    //     purifier.updateAirQuality(30.0);
    //     purifier.updateAirQuality(55.0);
    //     purifier.updateFilterUsage(200.0);
    //     purifier.updateFilterUsage(250.0);
    //     purifier.updateFilterUsage(-2.0);
    //     purifier.updateFilterUsage(550.0);
    //     purifier.updateAirQuality(55.0);

```

```
// testcase-UVT:
    purifier.updateAirQuality(80.0);
    purifier.updateAirQuality(30.0);
    purifier.updateAirQuality(55.0);
    purifier.updateFilterUsage(200.0);
    purifier.updateFilterUsage(250.0);
    purifier.updateFilterUsage(-2.0);
    purifier.updateFilterUsage(550.0);
    purifier.updateAirQuality(55.0);
}
}
```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c74_SmartAirPurifier.java	100.0 %	10	0	10
▼ c74_SmartAirPurifier	100.0 %	10	0	10
main(String[])		0	0	0
c74_SmartAirPurifier(double, do		0	0	0
adjustFanSpeed()	100.0 %	4	0	4
updateAirQuality(double)	100.0 %	2	0	2
updateFilterUsage(double)	100.0 %	4	0	4

3. Vibration testing instruction coverage: 96.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	159	20,828	20,987
▼ src	0.8 %	159	20,828	20,987
▼ code	0.8 %	159	20,828	20,987
▼ c74_SmartAirPurifier.java	96.4 %	159	6	165
▼ c74_SmartAirPurifier	96.4 %	159	6	165
main(String[])	100.0 %	34	0	34
c74_SmartAirPurifier(double, do	100.0 %	18	0	18
adjustFanSpeed()	100.0 %	38	0	38
updateAirQuality(double)	100.0 %	33	0	33
updateFilterUsage(double)	100.0 %	36	0	36

4. Fuzz testing branch coverage: 40%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c74_SmartAirPurifier.java	40.0 %	4	6	10
▼ c74_SmartAirPurifier	40.0 %	4	6	10
main(String[])		0	0	0
c74_SmartAirPurifier(double, do		0	0	0
adjustFanSpeed()	75.0 %	3	1	4
updateAirQuality(double)	50.0 %	1	1	2

5. Fuzz testing instruction coverage: 62.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	103	20,884	20,987
▼ src	0.5 %	103	20,884	20,987
▼ code	0.5 %	103	20,884	20,987
▼ c74_SmartAirPurifier.java	62.4 %	103	62	165
▼ c74_SmartAirPurifier	62.4 %	103	62	165
main(String[])	100.0 %	34	0	34
c74_SmartAirPurifier(double, do	100.0 %	18	0	18
adjustFanSpeed()	55.3 %	21	17	38
updateAirQuality(double)	90.9 %	30	3	33

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c74_SmartAirPurifier.java	100.0 %	10	0	10
▼ c74_SmartAirPurifier	100.0 %	10	0	10
main(String[])		0	0	0
c74_SmartAirPurifier(double, do		0	0	0
adjustFanSpeed()	100.0 %	4	0	4
updateAirQuality(double)	100.0 %	2	0	2
updateFilterUsage(double)	100.0 %	4	0	4

7. Z3 instruction coverage: 96.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	159	20,828	20,987
▼ src	0.8 %	159	20,828	20,987
▼ code	0.8 %	159	20,828	20,987
▼ c74_SmartAirPurifier.java	96.4 %	159	6	165
▼ c74_SmartAirPurifier	96.4 %	159	6	165
main(String[])	100.0 %	34	0	34
c74_SmartAirPurifier(double, do	100.0 %	18	0	18
adjustFanSpeed()	100.0 %	38	0	38
updateAirQuality(double)	100.0 %	33	0	33
updateFilterUsage(double)	100.0 %	36	0	36

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c74_SmartAirPurifier.java	100.0 %	10	0	10
▼ c74_SmartAirPurifier	100.0 %	10	0	10
main(String[])		0	0	0
c74_SmartAirPurifier(double, do		0	0	0
adjustFanSpeed()	100.0 %	4	0	4
updateAirQuality(double)	100.0 %	2	0	2
updateFilterUsage(double)	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 96.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	159	20,828	20,987
▼ src	0.8 %	159	20,828	20,987
▼ code	0.8 %	159	20,828	20,987
▼ c74_SmartAirPurifier.java	96.4 %	159	6	165
▼ c74_SmartAirPurifier	96.4 %	159	6	165
main(String[])	100.0 %	34	0	34
c74_SmartAirPurifier(double, do	100.0 %	18	0	18
adjustFanSpeed()	100.0 %	38	0	38
updateAirQuality(double)	100.0 %	33	0	33
updateFilterUsage(double)	100.0 %	36	0	36